基于Spark的地震数据处理与分析

https://dblab.xmu.edu.cn/blog/2738/

本案例针对全球重大地震数据进行分析，采用Python为编程语言，采用Hadoop存储数据，采用Spark对数据进行处理分析，并对结果进行数据可视化。

一、 实验环境搭建

（1）Linux：Ubuntu 16.04
（2）Hadoop：3.1.3（安装教程）
（3）Spark：2.4.0（安装教程）
（4）Python：Anaconda3（Anaconda及Jupyter Notebook安装教程）
（5）运行环境：Jupyter Notebook
（6）安装可视化所需软件包，在Linux终端下输入以下命令即可：

```
1. sudo apt-get install python3-matplotlib
2. sudo apt-get install python3-pandas
3. sudo apt-get install python3-
   mpltoolkits.basemap
```

Shell 命令

二、数据准备

数据来自和鲸社区的1965-2016全球重大地震数据，文件名为`earthquake.csv`包括23412条地震数据，其中有很多属性大部分缺失且本次实验用不到，将其手动删除，只保留Date, Time, Latitude, Longitude, Type, Depth, Magnitude这七个属性。

数据集下载百度网盘地址为：https://pan.baidu.com/s/1tI12CzrqXB8SrlgWokFMyg 提取码: 1ieo

由于数据集中存在3行有问题的数据，因此，需要手动进行如下修改：

	列 Date	列 Time
第 3380 行	1975-02-23T02:58:41.000Z ↓ 02/23/1975	1975-02-23T02:58:41.000Z ↓ 02:58:41
第 7514 行	1985-04-28T02:53:41.530Z ↓ 04/28/1985	1985-04-28T02:53:41.530Z ↓ 02:53:41
第 20652 行	2011-03-13T02:23:34.520Z ↓ 03/13/2011	2011-03-13T02:23:34.520Z ↓ 02:23:34

在Linux终端输入以下命令启动Hadoop，注意全程保持该终端开启。

```
cd /usr/local/hadoop
./sbin/start-dfs.sh
```

Shell 命令

使用Jupyter Notebook建立一个代码文件preprocessing.py，进行数据的预处理，内容如下：

```
from pyspark import SparkConf, SparkContext
from pyspark.sql import SparkSession
from pyspark.sql.functions import count, when,
split, posexplode

conf =
SparkConf().setMaster("local").setAppName("pre
processing")
sc = SparkContext(conf = conf)
sc.setLogLevel('WARN') # 减少不必要的log输出
spark = SparkSession.builder \
                    .config(conf =
SparkConf()) \
```

```
                    .getOrCreate()

rawFile = 'file:///home/hadoop/HBING/earthquake.csv'
rawData = spark.read.format('csv') \
                    .options(header='true', inferschema='true') \
                    .load(rawFile)
rawData.printSchema() # 查看数据结构
print('total count: %d' % rawData.count()) # 打印总行数
# 查看每列的非空行数
rawData.agg(*[count(c).alias(c) for c in rawData.columns]).show()

# 提取数据
newData = rawData.select('Date', 'Time', 'Latitude',
                         'Longitude', 'Type', 'Depth',
                         'Magnitude', 'Magnitude Type',
                         'ID', 'Source','Location Source',
                         'Magnitude Source', 'Status')
# 拆分'Date'到'Month',' Day', 'Year'
newData = newData.withColumn('Split Date', split(newData.Date, '/'))
attrs = sc.parallelize(['Month',' Day', 'Year']).zipWithIndex().collect()
for name, index in attrs:
    newColumn = newData['Split Date'].getItem(index)
    newData = newData.withColumn(name, newColumn)
newData = newData.drop('Split Date')
```

```
    newData.show(5)

    # 上传文件至HDFS
    newData.write.csv('earthquakeData.csv', header='true')
```
Python

在上述代码中，尽管数据集定义了很多数据特征，但是不少特征的缺失值较多，其数据过于稀疏，不利于进行分析。因此，在上述代码中，仅选择数据稠密的特征进行分析，并对Date列进行拆分。

上述代码运行结果如下：

```
root
 |-- Date: string (nullable = true)
 |-- Time: string (nullable = true)
 |-- Latitude: double (nullable = true)
 |-- Longitude: double (nullable = true)
 |-- Type: string (nullable = true)
 |-- Depth: double (nullable = true)
 |-- Depth Error: double (nullable = true)
 |-- Depth Seismic Stations: integer (nullable = true)
 |-- Magnitude: double (nullable = true)
 |-- Magnitude Type: string (nullable = true)
 |-- Magnitude Error: double (nullable = true)
 |-- Magnitude Seismic Stations: integer (nullable = true)
 |-- Azimuthal Gap: double (nullable = true)
 |-- Horizontal Distance: double (nullable = true)
 |-- Horizontal Error: double (nullable = true)
 |-- Root Mean Square: double (nullable = true)
 |-- ID: string (nullable = true)
 |-- Source: string (nullable = true)
 |-- Location Source: string (nullable = true)
 |-- Magnitude Source: string (nullable = true)
 |-- Status: string (nullable = true)
```

```
total count: 23412
+-----+-----+--------+---------+-----+-----+-----------+----------------------+---------+--------------+---------------+--------------------------+-------------+--------------------+----------------+--------------------+--+------+---------------+----------------+------+
| Date| Time|Latitude|Longitude| Type|Depth|Depth Error|Depth Seismic Stations|Magnitude|Magnitude Type|Magnitude Error|Magnitude Seismic Stations|Azimuthal Gap|Horizontal Distance|Horizontal Error|Root Mean Square|  ID|Source|Location Source|Magnitude Source|Status|
+-----+-----+--------+---------+-----+-----+-----------+----------------------+---------+--------------+---------------+--------------------------+-------------+--------------------+----------------+--------------------+--+------+---------------+----------------+------+
|23412|23412|   23412|    23412|23412|23412|       4461|                  7097|    23412|         23409|            327|                      2564|         7299|                1604|            1156|               17352|23412| 23412|          23412|           23412| 23412|
+-----+-----+--------+---------+-----+-----+-----------+----------------------+---------+--------------+---------------+--------------------------+-------------+--------------------+----------------+--------------------+--+------+---------------+----------------+------+
```

```
+----------+--------+--------+---------+-----------+-----+---------+--------------+--------------------+------+---------------+----------------+---------+-----+----+----+
|      Date|    Time|Latitude|Longitude|       Type|Depth|Magnitude|Magnitude Type|                  ID|Source|Location Source|Magnitude Source|   Status|Month| Day|Year|
+----------+--------+--------+---------+-----------+-----+---------+--------------+--------------------+------+---------------+----------------+---------+-----+----+----+
|01/02/1965|13:44:18|  19.246|  145.616|Earthquake|131.6|      6.0|            MW|ISCGEM860706|ISCGEM|         ISCGEM|          ISCGEM|Automatic|   01|  02|1965|
|01/04/1965|11:29:49|   1.863|  127.352|Earthquake| 80.0|      5.8|            MW|ISCGEM860737|ISCGEM|         ISCGEM|          ISCGEM|Automatic|   01|  04|1965|
|01/05/1965|18:05:58| -20.579| -173.972|Earthquake| 20.0|      6.2|            MW|ISCGEM860762|ISCGEM|         ISCGEM|          ISCGEM|Automatic|   01|  05|1965|
|01/08/1965|18:49:43| -59.076|  -23.557|Earthquake| 15.0|      5.8|            MW|ISCGEM860856|ISCGEM|         ISCGEM|          ISCGEM|Automatic|   01|  08|1965|
|01/09/1965|13:32:50|  11.938|  126.427|Earthquake| 15.0|      5.8|            MW|ISCGEM860890|ISCGEM|         ISCGEM|          ISCGEM|Automatic|   01|  09|1965|
+----------+--------+--------+---------+-----------+-----+---------+--------------+--------------------+------+---------------+----------------+---------+-----+----+----+
only showing top 5 rows
```

三、数据分析及可视化

1.全部代码

使用Jupyter Notebook编写数据分析代码文件analyze.py，内容如下：

```
from pyspark import SparkConf, SparkContext
from pyspark.sql import SparkSession
from pyspark.ml.feature import StringIndexer,
IndexToString
from pyspark.ml import Pipeline

import pandas as pd
import matplotlib.pyplot as plt
import mpl_toolkits.basemap

conf =
SparkConf().setMaster("local").setAppName("ana
lyze")
sc = SparkContext(conf = conf)
sc.setLogLevel('WARN') # 减少不必要的log输出
spark = SparkSession.builder.config(conf =
SparkConf()).getOrCreate()

dataFile = 'earthquakeData.csv'

rawData = spark.read.format('csv') \
                    .options(header='true',
inferschema='true') \
                    .load(dataFile)

def earthquakesPerYear():
    yearDF =
rawData.select('Year').dropna().groupBy(rawDat
a['Year']).count()
    yearDF =
yearDF.sort(yearDF['Year'].asc())
    yearPd = yearDF.toPandas()
    # 数据可视化
    plt.bar(yearPd['Year'], yearPd['count'],
color='skyblue')
    plt.title('earthquakes per year')
    plt.xlabel('Year')
    plt.ylabel('count')
```

```
    plt.show()

def earthquakesPerMonth():
    data = rawData.select('Year', 'Month').dropna()
    yearDF = data.groupBy(['Year', 'Month']).count()
    yearPd = yearDF.sort(yearDF['Year'].asc(), yearDF['Month'].asc()).toPandas()
    # 数据可视化
    x = [m+1 for m in range(12)]
    for groupName, group in yearPd.groupby('Year'):
        # plt.plot(x, group['count'], label=groupName)
        plt.scatter(x, group['count'], color='g', alpha=0.15)
    plt.title('earthquakes per month')
    # plt.legend() # 显示label
    plt.xlabel('Month')
    plt.show()

def depthVsMagnitude():
    data = rawData.select('Depth', 'Magnitude').dropna()
    vsPd = data.toPandas()
    # 数据可视化
    plt.scatter(vsPd.Depth, vsPd.Magnitude, color='g', alpha=0.2)
    plt.title('Depth vs Magnitude')
    plt.xlabel('Depth')
    plt.ylabel('Magnitude')
    plt.show()

def magnitudeVsType():
```

```
    data = rawData.select('Magnitude',
'Magnitude Type').dropna()
    typePd = data.toPandas()
    # 准备箱体数据
    typeBox = []
    typeName = []
    for groupName, group in
typePd.groupby('Magnitude Type'):
        typeName.append(groupName)
        typeBox.append(group['Magnitude'])
    # 数据可视化
    plt.boxplot(typeBox, labels=typeName)
    plt.title('Type vs Magnitude')
    plt.xlabel('Type')
    plt.ylabel('Magnitude')
    plt.show()

def location():
    data = rawData.select('Latitude',
'Longitude', 'Magnitude').dropna()
    locationPd = data.toPandas()
    # 世界地图
    basemap = mpl_toolkits.basemap.Basemap()
    basemap.drawcoastlines()
    # 数据可视化
    plt.scatter(locationPd.Longitude,
locationPd.Latitude,
                color='g', alpha=0.25,
s=locationPd.Magnitude)
    plt.title('Location')
    plt.xlabel('Longitude')
    plt.ylabel('Latitude')
    plt.show()

if __name__ == '__main__':
    earthquakesPerYear()
    # earthquakesPerMonth()
```

```
    # depthVsMagnitude()
    # magnitudeVsType()
    # location()
```
Python

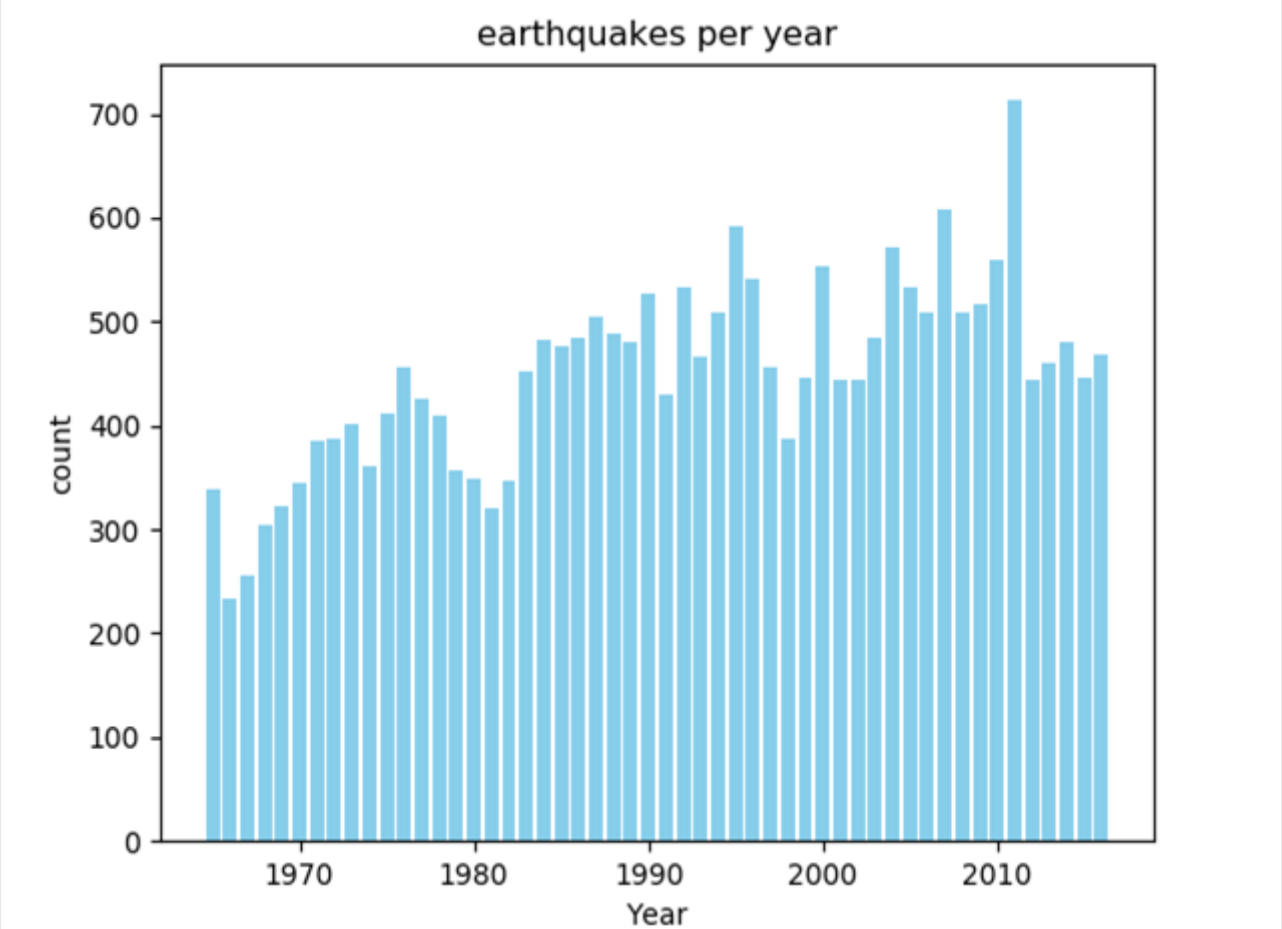
上面是分析的全部代码，下面我们分开讲解每个部分。

2.全球每年发生重大地震的次数

这个部分对应的代码如下：

```
def earthquakesPerYear():
    yearDF = rawData.select('Year').dropna() \
                    .groupBy(rawData['Year']).
count()
    yearDF = yearDF.sort(yearDF['Year'].asc())
    yearPd = yearDF.toPandas()
    # 数据可视化
    plt.bar(yearPd['Year'], yearPd['count'],
color='skyblue')
    plt.title('earthquakes per year')
    plt.xlabel('Year')
    plt.ylabel('count')
    plt.show()
```
Python

可视化结果如下图所示。由图可知，1965-2016年期间，每年发生重大地震的次数均超过200次，且总体趋势上，随着年份的增加地震发生的频率也提高了，从2012年开始有有所回落，但每年发生地震次数仍达到400以上。



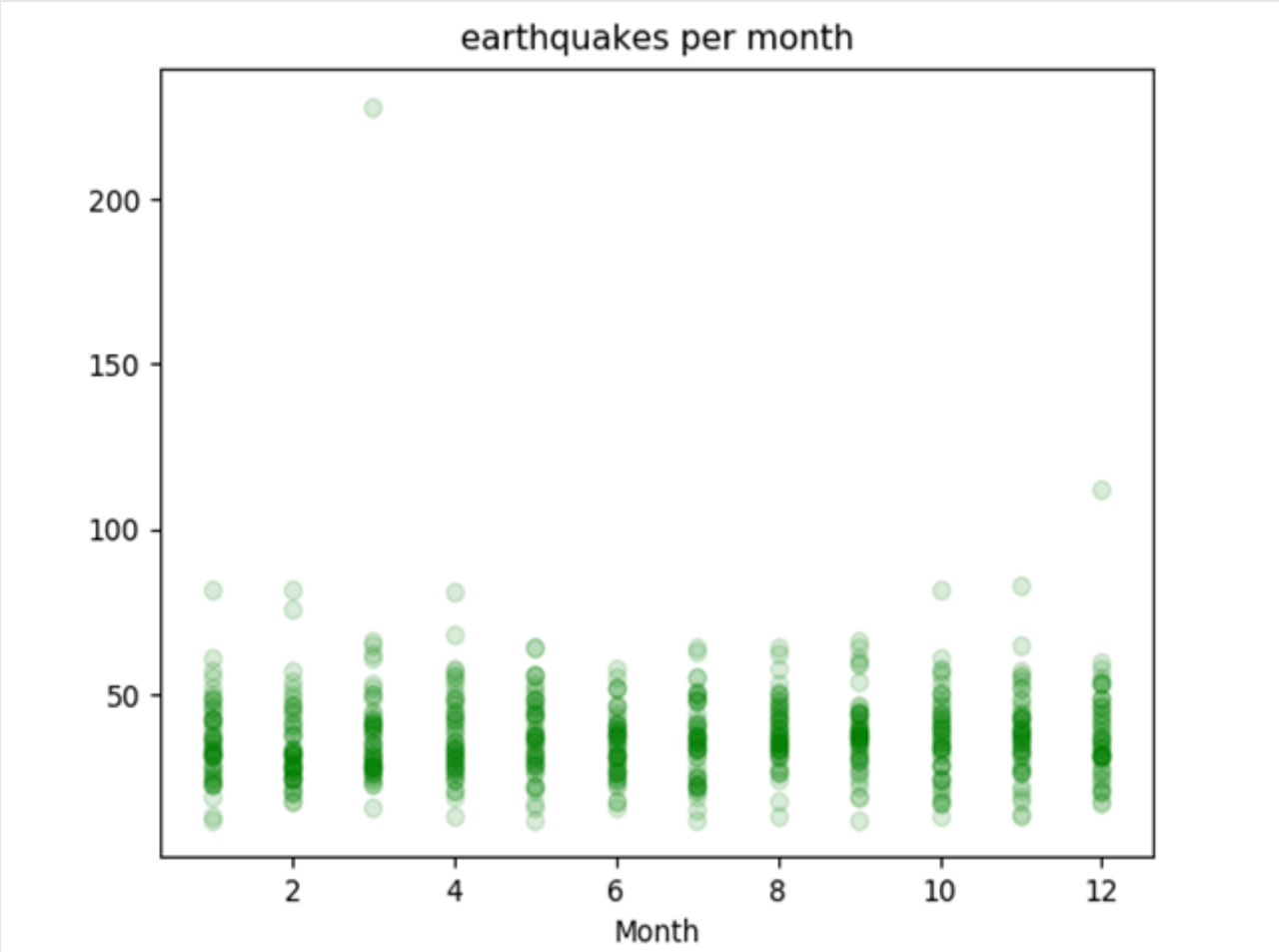
2.全球不同年份每月发生重大地震的次数

这个部分对应的代码如下：

```
def earthquakesPerMonth():
    data = rawData.select('Year', 'Month').dropna()
    yearDF = data.groupBy(['Year', 'Month']).count()
    yearPd = yearDF.sort(yearDF['Year'].asc(), yearDF['Month'].asc()).toPandas()
    # 数据可视化
    x = [m+1 for m in range(12)]
    for groupName, group in yearPd.groupby('Year'):
        # plt.plot(x, group['count'], label=groupName)
```

```
        plt.scatter(x, group['count'],
color='g', alpha=0.15)
    plt.title('earthquakes per month')
    # plt.legend() # 显示label
    plt.xlabel('Month')
    plt.show()
```
Python

可视化结果如下图所示。从图中我们可以看出，每月不同年份发生地震的次数基本都在 [10, 60] 这一区间，仅有个别年份的个别月份地震发生频率激增，最高超过200次。



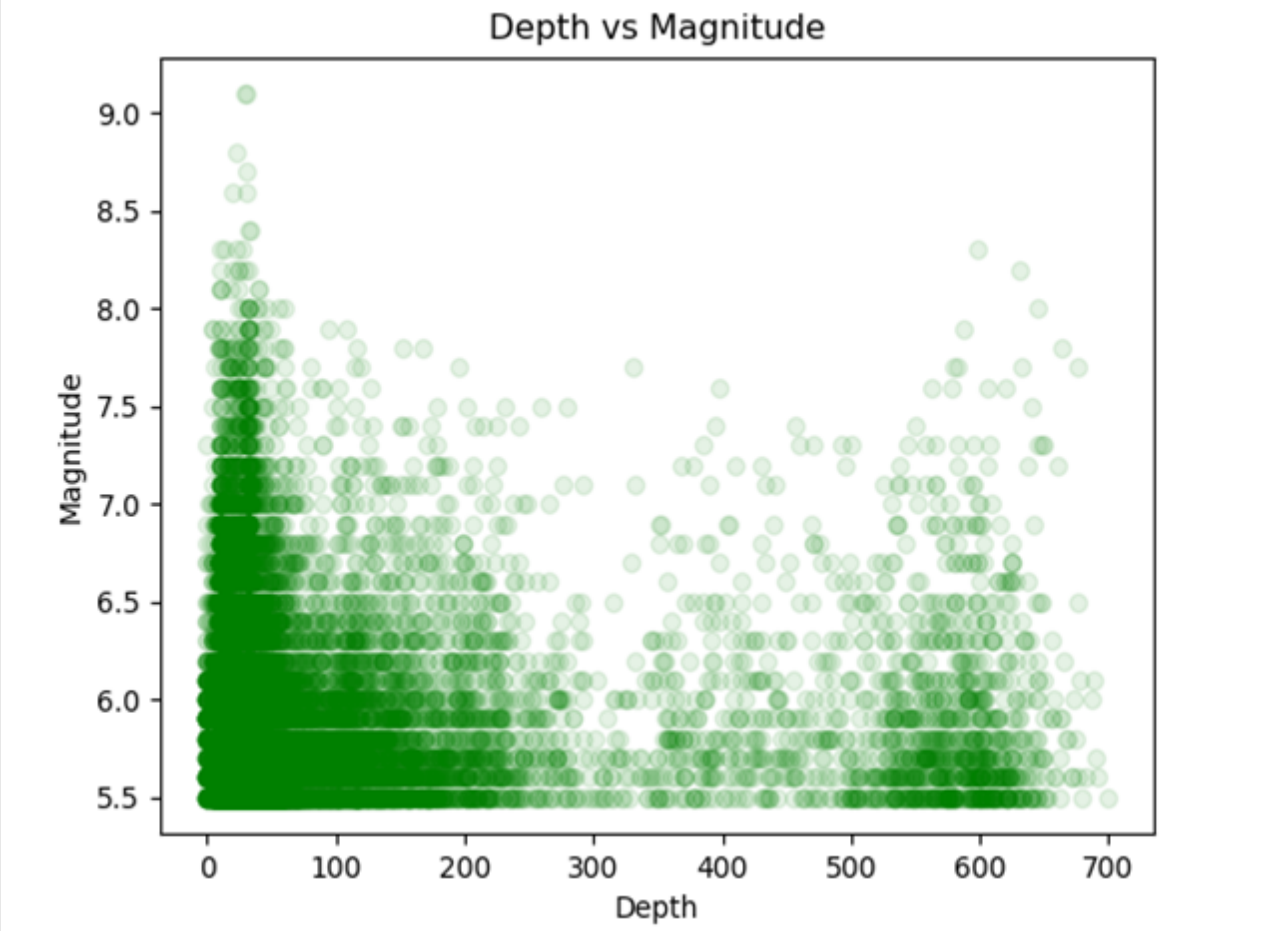
3.全球重大地震深度和强度之间的关系

这个部分对应的代码如下：

```
def depthVsMagnitude():
    data = rawData.select('Depth',
'Magnitude').dropna()
```

```
    vsPd = data.toPandas()
     # 数据可视化
    plt.scatter(vsPd.Depth, vsPd.Magnitude,
color='g', alpha=0.1)
    plt.title('Depth vs Magnitude')
    plt.xlabel('Depth')
    plt.ylabel('Magnitude')
    plt.show()
```
Python

可视化结果如下图所示。在图中，地震深度和强度并不存在明显的线性关系，在 [250, 500] 这一区间内，地震的强度和频率明显少于其他深度区间。反而在深度约60以下为地震高发区，且震级强度也分布较广。

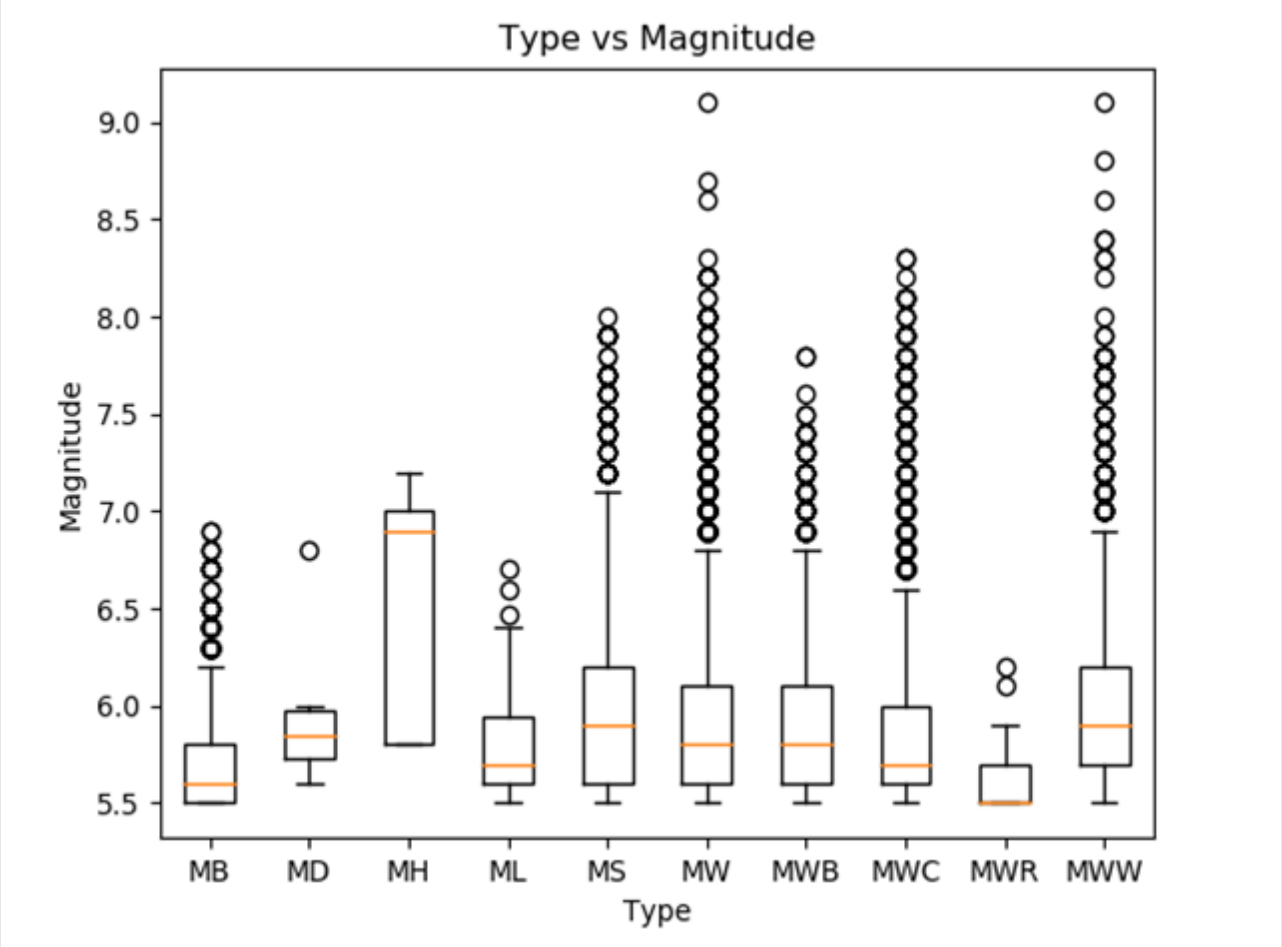


4.全球重大地震深度和类型之间的关系

这个部分对应的代码如下：

```Python
def magnitudeVsType():
    data = rawData.select('Magnitude', 'Magnitude Type').dropna()
    typePd = data.toPandas()
    #准备箱体数据
    typeBox = []
    typeName = []
    for groupName, group in typePd.groupby('Magnitude Type'):
        typeName.append(groupName)
        typeBox.append(group['Magnitude'])
    #数据可视化
    plt.boxplot(typeBox, labels=typeName)
    plt.title('Type vs Magnitude')
    plt.xlabel('Type')
    plt.ylabel('Magnitude')
    plt.show()
```

可视化结果如下图所示。由图可知，MH类型的地震强度普遍较其他类型更高，主要分布于 [5.7, 7.0] 之间，而MB、MS、MW、MWB、MWC和MWW类型的地震数据出现了较多的异常点，可能由于数据不服从正态分布存在较高的偏态。



5.全球经常发生重大地震的地带

这个部分对应的代码如下：

```
def location():
    data = rawData.select('Latitude',
                                'Longitude',

'Magnitude').dropna()
    locationPd = data.toPandas()
    # 世界地图
    basemap = mpl_toolkits.basemap.Basemap()
    basemap.drawcoastlines()
    # 数据可视化
     plt.scatter(locationPd.Longitude,
locationPd.Latitude,
                    color='g', alpha=0.25,
s=locationPd.Magnitude)
```

```
    plt.title('Location')
    plt.xlabel('Longitude')
    plt.ylabel('Latitude')
    plt.show()
```
Python

可视化结果如下图所示。由图中可明显看出不同的地震带把地球明显分成好几个板块，环太平洋火山地震带上，地震发生频率和震级都较其他地震带要高。

